

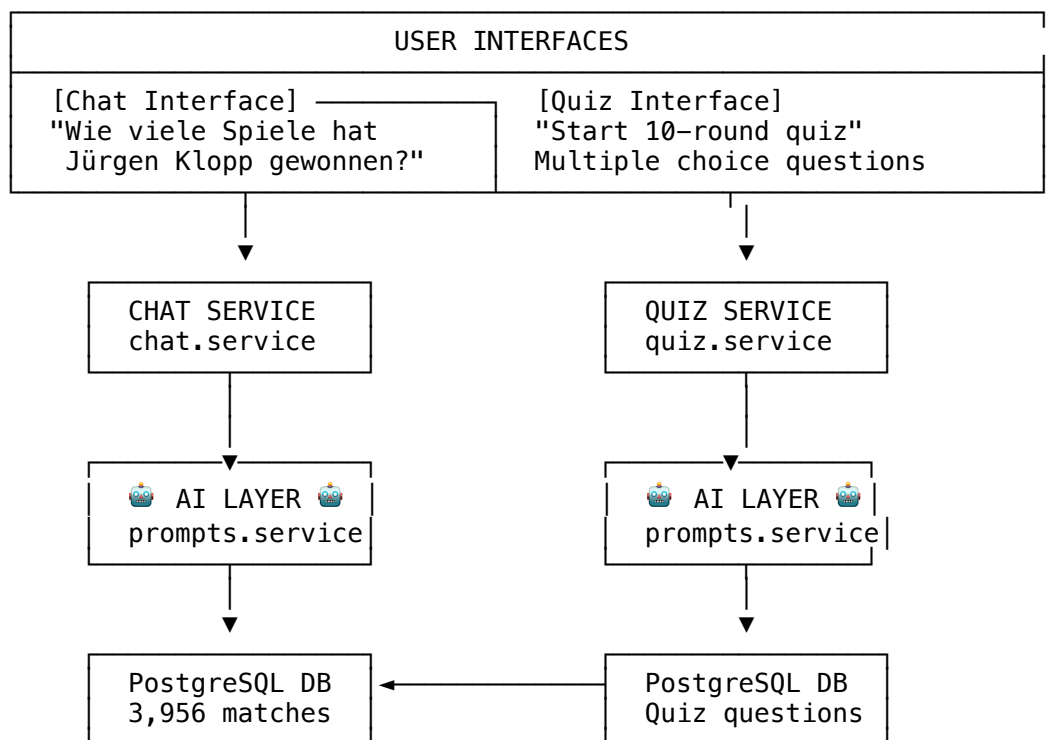
FSV Mainz 05 Archive - AI Integration Flow

Focus: Where AI/Prompts are used and what they accomplish

Last Updated: November 10, 2025

System Overview

The FSV Mainz 05 Archive uses AI at strategic points to transform structured data into natural language interactions. The system has **2 main user flows**: **Chat** and **Quiz**.



🎯 FLOW 1: Chat System (Natural Language → SQL → Answer)

User Journey

User Question → AI SQL Generation → Query Execution → AI Answer Formatting → Natural Response

Detailed Flow with AI Integration Points





2. 🚢 AI STEP 1: SQL GENERATION

Prompt: chat-sql-generator.txt

Model: qwen/qwen-2.5-72b-instruct (via OpenRouter)

INPUTS:

- userQuestion: "Wie viele Spiele hat Jürgen Klopp..."
- conversationHistory: [last 6 messages]
- schemaContext: Full database schema (tables, columns, relationships)

WHAT AI DOES:

- Understand user intent from German natural language
- Map question to database schema
- Generate syntactically correct PostgreSQL query
- Determine confidence level (0-100)
- Identify if clarification is needed

OUTPUT FORMAT (JSON):

```
{
  "sql": "SELECT c.name, COUNT(DISTINCT m.match_id)...",
  "confidence": 95,
  "needsClarification": null,
  "explanation": "Query counts wins where Klopp was coach..."
}
```

PROMPT ENGINEERING FOCUS:

- Schema awareness (knows teams, matches, coaches tables)
- German language understanding
- Handling ambiguity (player vs coach, wins vs draws)
- Date/time period recognition
- Team name normalization (Mainz 05, FSV Mainz, etc.)

OBSERVABILITY:

- Tracked in Langfuse: trace "chat-sql-generation"



3. VALIDATION & CLARIFICATION CHECK

IF confidence < threshold OR needsClarification:

- Return clarification question to user
- "Meinst du Klopp als Spieler oder als Trainer?"

ELSE:

- Continue to SQL execution



4. SQL EXECUTION (NO AI)

postgres.service.executeUserQuery(sql)

Security:

- Read-only transaction (BEGIN TRANSACTION READ ONLY)
- 10-second timeout
- Row limit (max 1000 results)

Metrics Collected:

- Execution time (ms)
- Row count

EXAMPLE OUTPUT:

```
[
  { name: "JÜRGEN KLOPP", win_count: 185, total_matches: 431 },
  ...
]
```



5. 🤖 AI STEP 2: ANSWER FORMATTING

Prompt: chat-answer-formatter.txt

Model: qwen/qwen-2.5-72b-instruct (via OpenRouter)

INPUTS:

```
└ userQuestion: Original question
└ sqlQuery: Generated SQL
└ sqlResult: Query result rows
└ resultMetadata: { rowCount: 1, executionTimeMs: 45 }
```

WHAT AI DOES:

```
└ Convert tabular data to natural German language
└ Add context and insights
└ Highlight interesting facts
└ Suggest visualizations
└ Generate follow-up questions
```

OUTPUT FORMAT (JSON):

```
{
  "answer": "Jürgen Klopp hat als Trainer von Mainz 05...",
  "highlights": [
    "185 Siege in 431 Spielen (42.9% Siegquote)",
    "Längste Amtszeit: 2001-2008"
  ],
  "suggestedVisualization": "bar_chart",
  "followUpQuestions": [
    "Wie schnitt Klopp im Vergleich zu anderen Trainern ab?",
    "In welcher Saison hatte Klopp die meisten Siege?"
  ]
}
```

PROMPT ENGINEERING FOCUS:

- German language fluency
- Mainz 05 fan tone (enthusiastic but professional)
- Statistical accuracy (no hallucination)
- Contextual enrichment (e.g., "in der 2. Bundesliga")
- Smart follow-ups (related but different questions)

OBSERVABILITY:

```
└ Tracked in Langfuse: trace "chat-answer-formatting"
```



6. RESPONSE TO USER

Display formatted answer with:

- ✓ Natural language response
- ✓ Key highlights (bullet points)
- ✓ Suggested follow-up questions (chips)
- ✓ Optional visualization

Metadata saved to database:

```
└ SQL query (for debugging)
└ Execution time
```

```
| Confidence score  
| Langfuse trace ID (for analysis)  
|
```

🎯 FLOW 2: Quiz System (Generate Questions & Validate Answers)

User Journey

Quiz Start → AI Question Generation → User Answers → AI Answer Validation → Scores

Detailed Flow with AI Integration Points

1. QUIZ CREATION REQUEST

User selects:

```
| Topic: "FSV Mainz 05 Geschichte" or "Spielerstatistiken"  
| Difficulty: easy / medium / hard  
| Number of rounds: 5-20  
| Player names: ["Anna", "Max"]
```

Database creates:

```
| quiz_games record (status: "creating")
```



2. 🤖 AI STEP 1: QUESTION GENERATION (per round)

Prompt: quiz-question-generator.txt

Model: qwen/qwen-2.5-72b-instruct (via OpenRouter)

Frequency: Called N times (N = number of rounds)

INPUTS:

```
| category: "statistics" | "history" | "players" | "matches"  
| difficulty: "easy" | "medium" | "hard"  
| schemaContext: Database schema understanding  
| previousQuestions: [already generated questions] (avoid duplicates)
```

WHAT AI DOES:

```
| Create interesting, factual question about Mainz 05  
| Generate 4 answer options (1 correct, 3 plausible distractors)  
| Ensure question is answerable from database  
| Match difficulty level  
| Avoid repetition with previous questions
```

OUTPUT FORMAT (JSON):

```
{  
  "question_text": "Wer erzielte das erste Tor im neuen Stadion?",  
  "options": [  
    { "text": "André Schürrle", "is_correct": true },  
    { "text": "Mohamed Zidan", "is_correct": false },  
    { "text": "Eugen Polanski", "is_correct": false },  
    { "text": "Lewis Holtby", "is_correct": false }  
  ],  
  "explanation": "Schürrle erzielte das Tor in der 12. Minute...",  
  "category": "history",  
  "difficulty": "medium",  
  "points": 100  
}
```

PROMPT ENGINEERING FOCUS:

- Difficulty calibration:
 - Easy: "Wer ist der aktuelle Trainer?" (common knowledge)
 - Medium: "Wie viele Tore schoss Schürrle 2009/10?" (stats)
 - Hard: "Gegen wen spielte Mainz am 15.03.1997?" (deep history)
- Plausible distractors (wrong answers must seem reasonable)
- Factual accuracy (verifiable from database)
- Variety (mixing player stats, match history, records)
- German language (natural phrasing)

OBSERVABILITY:

- └ Tracked in Langfuse: trace "quiz-question-generation"

**3. QUESTION STORAGE & DISPLAY****Database saves:**

- └ quiz_rounds (game_id, round_number, status)
- └ quiz_questions (question_text, correct_answer_id)

Frontend displays:

- └ Question text
- └ 4 multiple choice options
- └ Timer (if timed mode)
- └ Round progress (3/10)

**4. USER SUBMITS ANSWER**

User clicks option: "André Schürrle"

Backend receives:

- └ game_id
- └ player_name
- └ selected_answer_id
- └ time_taken_ms

**5. 🤖 AI STEP 2: ANSWER VALIDATION & EXPLANATION**

Prompt: quiz-answer-generator.txt

Model: qwen/qwen-2.5-72b-instruct (via OpenRouter)

INPUTS:

- └ question: Original question object
- └ userAnswer: Selected option
- └ correctAnswer: Correct option
- └ isCorrect: boolean

WHAT AI DOES:

- └ Generate contextual explanation
- └ Add interesting facts
- └ Provide encouragement (if wrong) or praise (if correct)
- └ Link to related history/stats

OUTPUT FORMAT (JSON):

```
{
  "explanation": "Richtig! Schürrle erzielte das erste Tor...",
  "additionalInfo": "Schürrle spielte von 2009-2011 bei Mainz..."
}
```

```
"funFact": "Dieses Spiel war vor 32.000 Zuschauern!",  
"relatedStat": "Schürle: 27 Tore in 65 Spielen"  
}
```

PROMPT ENGINEERING FOCUS:

- Encouraging tone (even when answer is wrong)
- Educational value (teach something new)
- Mainz 05 trivia (interesting context)
- German language (conversational)

OBSERVABILITY:

└ Tracked in Langfuse: trace "quiz-answer-generation"

**6. SCORING & FEEDBACK****Calculate points:**

- └ Base points (100 for easy, 200 for medium, 300 for hard)
- └ Time bonus (faster = more points)
- └ Streak multiplier (consecutive correct answers)

Update database:

- └ quiz_answers (player_name, is_correct, points_earned)
- └ quiz_rounds (status: "completed")

Display to user:

- └ ✓ / ✗ Correct or Wrong
- └ Points earned
- └ AI-generated explanation
- └ Current score

 Technical Architecture: AI Components**1. Prompt Management System****PROMPTS SERVICE (apps/api/src/services/ai/prompts.service.ts)****Responsibilities:**

- └ Load prompt templates from Langfuse or local fallback
- └ Compile templates with variables (schema, user input)
- └ Create Langfuse traces for observability
- └ Call OpenRouter API
- └ Log generation metrics (latency, tokens, cost)

Prompt Sources (priority order):

1. Langfuse (remote, versioned, A/B testable)
2. Local fallback (prompts/fallback/*.txt)

Prompt Template Format:**SYSTEM INSTRUCTION:**

You are an expert SQL generator for FSV Mainz 05...
{{schemaContext}}

```
USER PROMPT:
User question: {{userQuestion}}
Conversation history: {{conversationHistory}}
```

Available Prompts:

- └ chat-sql-generator.txt (Question → SQL)
- └ chat-answer-formatter.txt (SQL Result → Natural Language)
- └ quiz-question-generator.txt (Generate quiz questions)
- └ quiz-answer-generator.txt (Explain quiz answers)

2. LLM Integration (OpenRouter)

OPENROUTER SERVICE (apps/api/src/services/ai/openrouter.service)

Model: qwen/qwen-2.5-72b-instruct

Why: Best German language support + JSON mode + cost-effective

Configuration:

- └ Temperature: 0.1 (low for factual accuracy)
- └ Max tokens: 2000
- └ Response format: JSON (structured output)
- └ Timeout: 30 seconds

API Features Used:

- └ System instructions (role context)
- └ JSON schema validation (type safety)
- └ Streaming (future: real-time responses)

Error Handling:

- └ Retry logic (3 attempts with exponential backoff)
- └ Fallback to cached responses
- └ Graceful degradation (return error message to user)

3. Observability (Langfuse)

LANGFUSE SERVICE (apps/api/src/services/ai/langfuse.service)

Purpose: Track AI performance, costs, and quality

Tracked Metrics:

- └ Latency per generation (ms)
- └ Token usage (input + output)
- └ Cost per request (\$)
- └ User feedback (thumbs up/down)
- └ Error rates

Trace Structure:

Trace (chat-sql-generation)

- └ Generation (openrouter-sql-generation)
 - └ Input: { system, user }
 - └ Output: { sql, confidence }
 - └ Latency: 450ms
 - └ Tokens: { input: 1200, output: 150 }

Use Cases:

- └ Debug low-confidence SQL generations
- └ A/B test different prompt versions
- └ Monitor cost per user interaction
- └ Identify and fix hallucinations

Prompt Engineering Guidelines

What Makes a Good Prompt in This System?

1. Schema Awareness

✗ BAD: "You are a SQL generator."

✓ GOOD: "You are a SQL generator with access to FSV Mainz 05 archive.
Key tables: teams, players, matches, goals, coaches.
Team ID 1 = FSV Mainz 05 (home team).
All dates are in ISO format (YYYY-MM-DD)."

2. German Language Context

✗ BAD: "Answer the user's question."

✓ GOOD: "Answer in German (informal 'du' form, enthusiastic Mainz fan tone).
Use '05er' slang when appropriate.
Example: 'Die Nullfünfer haben...' not 'FSV Mainz 05 hat...'"

3. Factual Grounding

✗ BAD: "Make up interesting facts."

✓ GOOD: "CRITICAL: Only use data from the SQL result. DO NOT hallucinate.
If data is missing, say 'Diese Information ist nicht verfügbar.'
Verify all numbers match the query result exactly."

4. Output Structure

✗ BAD: "Generate a question."

✓ GOOD: "Generate a question in this EXACT JSON format:
{
 'question_text': string (German, ends with '?'),
 'options': array of 4 objects with 'text' and 'is_correct',
 'explanation': string (2-3 sentences),
 'difficulty': 'easy' | 'medium' | 'hard'
}
Validate: Exactly 1 correct answer, all options are plausible."

5. Difficulty Calibration (Quiz)

EASY prompts should:

- └ Ask about current/recent events (last 2 seasons)
- └ Reference famous players everyone knows
- └ Have obvious distractors

MEDIUM prompts should:

- └ Require statistical knowledge (goal counts, win ratios)
- └ Reference historical events (5–10 years ago)
- └ Have plausible but incorrect distractors

HARD prompts should:

- └ Deep historical knowledge (20+ years ago)
- └ Obscure statistics (records, specific dates)
- └ Very similar distractors (e.g., 27 goals vs 29 goals)



Monitoring & Optimization

Key Metrics to Track

METRIC	TARGET	ALERT THRESHOLD
SQL Generation Latency	< 500ms	> 1000ms
SQL Execution Latency	< 100ms	> 500ms
Answer Format Latency	< 800ms	> 1500ms
End-to-End Chat Latency	< 2s	> 5s
SQL Confidence Score	> 80%	< 60%
Quiz Question Uniqueness	100%	< 95% (duplicates)
User Satisfaction	> 80%	< 70% (thumbs up)
Cost per Chat Interaction	< \$0.005	> \$0.02
SQL Error Rate	< 5%	> 15%

Optimization Strategies

When SQL Confidence is Low:

1. Schema Context Too Large?

- Reduce to relevant tables only based on question
- Use semantic search to find relevant schema parts

2. Ambiguous Questions?

- Add examples of ambiguous vs clear questions to prompt
- Include clarification decision tree

3. Poor Prompt?

- A/B test variations in Langfuse
- Add more few-shot examples

When Answers Hallucinate:

1. Prompt Too Creative?

- Lower temperature (0.1 → 0.0)
- Add “DO NOT ADD INFORMATION NOT IN RESULT” in bold

2. Result Format Unclear?

- Show exact SQL result format in prompt
- Add example transformations



Future AI Enhancements

Planned Improvements

1. Semantic Search for Schema

Current: Send full schema to LLM (8000 tokens)

Future: Embed schema chunks, retrieve top 5 relevant tables (1000 tokens)

Benefit: 50% faster, 70% cheaper

2. Query Cache with Semantic Matching

Current: Every question generates new SQL

Future: Check if similar question was asked before

"Wie viele Tore hat Schürrle geschossen?"

≈ "Schürrle Tore gesamt?"

→ Use cached SQL, just update parameters

Benefit: 90% faster for common questions

3. Multi-Step Reasoning for Complex Questions

Question: "Wer hat die meisten Tore in Spielen geschossen,
die wir mit mehr als 2 Toren Unterschied gewonnen haben?"

Current: Single prompt → complex SQL (often fails)

Future: Chain of Thought

Step 1: "Find matches won by 2+ goals"

Step 2: "Find goal scorers in those matches"

Step 3: "Aggregate and rank"

→ Generate sub-queries → Combine

4. User Intent Classification

Before generating SQL, classify intent:

- └ statistics (COUNT, AVG, MAX) → Use optimized stat queries
- └ history (specific match/date) → Use indexed lookups
- └ comparison (player A vs B) → Use materialized views
- └ exploration (open-ended) → Guide with follow-ups

5. Personalized Difficulty

Quiz: Track user performance

- └ If 80%+ correct → Increase difficulty automatically
- └ If < 50% correct → Decrease difficulty
- └ Learn user preferences (prefers player stats vs history)

File Reference

Key Files for AI Integration

```
apps/api/src/
├── services/
│   ├── chat/
│   │   └── chat.service.ts           # Main chat flow (2 AI calls)
│   ├── quiz/
│   │   └── quiz.service.ts          # Quiz flow (2 AI calls per round)
│   └── ai/
```

```

|      | prompts.service.ts      # Prompt loading & execution
|      | openrouter.service.ts  # LLM API calls
|      | langfuse.service.ts    # Observability tracking
|
prompts/fallback/
| chat-sql-generator.txt        # Question → SQL
| chat-answer-formatter.txt     # SQL Result → Natural Language
| quiz-question-generator.txt   # Generate quiz questions
| quiz-answer-generator.txt     # Explain quiz answers

```

✓ Summary: Where AI is Used

User Flow	AI Step	Purpose	Prompt
Chat	SQL Generation	Natural language → SQL query	chat-sql-generator
Chat	Answer Formatting	SQL result → Natural German response	chat-answer-formatter
Quiz (per round)	Question Generation	Create interesting quiz question	quiz-question-generator
Quiz (per answer)	Answer Explanation	Explain correct/incorrect answer	quiz-answer-generator

Total AI Calls per User Interaction: - Chat: **2 AI calls** (SQL gen + Answer format) - Quiz: **2N AI calls** (N rounds × 2 per round)

No AI Used For: - SQL execution (database query) - User authentication - Session management - Scoring calculation - Data parsing/ingestion

End of Document